

Desenvolvimento para Dispositivos Móveis

Banco de dados SQLite

Universidade Federal do Rio Grande do Norte

Banco de Dados

- Persistir grandes quantidades de dados é uma atividade comum em diversas aplicações.
- Dispositivos Mobile possuem nativamente o suporte a um banco de dados
 - SQLite
 - Banco de dados leve e poderoso
- Cada aplicação pode criar um ou mais bancos de dados.

Banco de Dados

- Os aplicativos Flutter podem fazer uso de bancos de dados SQLite por meio do plugin sqflite e path disponíveis no pub.dev.
 - O pacote sqflite fornece classes e funções para interagir com um banco de dados SQLite.
 - O pacote path fornece funções para definir a localização de armazenamento do banco de dados no disco.

```
flutter pub add sqflite path
```

- Embora essas APIs sejam poderosas, elas são de nível razoavelmente baixo e exigem muito tempo e esforço para usar.

Uso de SQLite demanda cuidados

- **Não há verificação em tempo de compilação de consultas SQL.**
- A medida que o banco de dados é alterado, é **necessário atualizar as consultas SQL afetadas manualmente**. Esse processo pode ser demorado e propenso a erros.
- O desenvolvedor acaba usando **muitos códigos repetidos para converter entre consultas SQL e objetos de dados**.

Definir Schemas e Contratos

- Antes de aprofundar um pouco mais na criação de bancos de dados é preciso lembrar de boas práticas relacionadas ao uso de bancos de dados
 - Schemas e Contratos

Schemas

- Um dos principais princípios dos bancos de dados SQL é o schema.
- Um schema é a declaração formal de como o banco de dados é organizado.
- O schema é refletido nas instruções SQL usadas para criar o banco de dados.

Definir Schemas e Contratos

- Antes de aprofundar um pouco mais na criação de bancos de dados é preciso lembrar de boas práticas relacionadas ao uso de bancos de dados
 - Schemas e Contratos

Contratos

- É aconselhável criar uma classe de acompanhamento, conhecida como classe de contrato.
- Contrato especifica explicitamente o layout do schema de forma sistemática e auto documentada.

Contratos

- Um conjunto de constantes que definem nomes para URIs, tabelas e colunas que pode ser definido
- Permite usar mesmas constantes em todas as outras partes do código.
- Isso permite que o nome de uma coluna alterado em um lugar seja propagado todo o código.

Classe Livro

```
class Livro {  
    int id = 0;  
    String titulo = '';  
    String autor = '';  
    int anoPublicacao = 0;  
    double avaliacao = 0;  
}
```

```
abstract class LivroContract {  
    static const String livroTable = "livro_table";  
    static const String idColumn = "id";  
    static const String tituloColumn = "titulo";  
    static const String autorColumn = "autor";  
    static const String anoPublicacaoColumn = "ano_publicacao";  
    static const String avaliacaoColumn = "avaliacao";  
}
```

Banco de dados SQLite

- Para facilitar o uso do SQLite o Flutter provê uma função chamada **openDatabase()**
- É necessário definir o caminho para o arquivo do banco de dados usando o `getDatabasesPath()` do pacote `sqflite`, combinado com a função `join` do pacote `path`.

```
final database = openDatabase(  
    join(await getDatabasesPath(), 'banco_de_dados.db'),  
);
```

Banco de dados SQLite

- O método `openDatabase` também dá a possibilidade de realizar ações/funções no momento da criação, `update` ou `downgrade` do banco de dados.
- Implementar pelo menos o método `OnCreate` para garantir que uma tabela seja criada.

Implementação

```
Future<Database> initDb() async {
    String? databasesPath = await getDatabasesPath();
    if (databasesPath == null) databasesPath = "";
    String path = join(databasesPath, "livros.db");

    return await openDatabase(path, version: 1,
        onCreate: (Database db, int newerVersion) async {
            await db.execute(
                "CREATE TABLE ${LivroContract.livroTable}(${LivroContract.idColumn} INTEGER PRIMARY KEY
AUTOINCREMENT, "
                "                                ${LivroContract.tituloColumn} TEXT, "
                "                                ${LivroContract.autorColumn} TEXT, "
                "                                ${LivroContract.anoPublicacaoColumn} INTEGER, "
                "                                ${LivroContract.avaliacaoColumn} REAL) ");
        });
}
```

Banco de dados SQLite

- Depois de criar esse template básico, dentro da classe você pode criar os métodos que quiser
- A lógica consiste em obter o objeto do Banco de Dados e realizar a operação desejada.

```
Database? _db;
```

```
Future<Database?> get db async {  
    if (_db == null) _db = await initDb();  
    return _db;  
}
```

Banco de dados SQLite

- O SQLite possui tipos para os seus dados de maneira semelhante a qualquer base de dados:
 - NULL
 - INTEGER
 - REAL
 - TEXT
 - BLOB

Implementação de CRUD

- Com um objeto **SQLiteDatabase** podemos sobrescrever e implementar os métodos:
 - insert
 - update
 - delete
 - query
 - execSQL

Inserção de dados

- Para inserir registros no banco de dados é necessário criar um objeto do tipo **Map**.
 - Contém uma estrutura simples de **chave e valor**.
 - Cada chave precisa corresponder **ao nome de uma coluna na tabela**.
 - Use o contrato para facilitar.
- `insert(nome_tabela, map)`

Inserção de Dados

```
Future<Livro> saveLivro(Livro l) async {  
    Database? dbLivro = await db;  
    if (dbLivro != null) {  
        l.id = await dbLivro.insert(LivroContract.LivroTable, l.toMap());  
    }  
    return l;  
}
```

Atualização de dados

- Devemos usar o método update
- `update(tabela, map, selection, whereArgs)`
- O parâmetro do selection é uma string para identificar o registro
- O parâmetro whereArgs são os dados para comparação dos dados a partir do selection.

Atualização de dados

```
Future<int> updateLivro(Livro l) async {  
    Database? dbLivro = await db;  
    if (dbLivro != null) {  
        return await dbLivro.update(LivroContract.LivroTable, l.toMap(),  
            where: "${LivroContract.idColumn} = ?", whereArgs: [l.id]);  
    } else {  
        return 0;  
    }  
}
```

Atualização de dados

Selection e whereArgs

- Note na string Selection a presença de um coringa “?”.
- Para cada coringa adicionado, um novo valor no array de strings whereArgs deve ser especificado.

Exclusão de dados

- Devemos usar o método delete(tabela, where, whereArgs)
- O parâmetro do where é uma string para identificar o registro.

```
Future<int> deleteLivro(int id) async {  
    Database? dbLivro = await db;  
    if (dbLivro != null) {  
        return await dbLivro.delete(LivroContract.LivroTable,  
            where: "${LivroContract.idColumn} = ?", whereArgs: [id]);  
    } else  
        return 0;  
}
```

Busca de Dados

- Para consultar dados de uma tabela é necessário usar o método `query` informando as colunas desejadas e a cláusula `where`
- O retorno é um objeto do tipo `List de Maps`

```
db.query(tabela:String, colunas: String[], seleção: String[],  
selecaoArgs: String[], groupBy:String, orderBy:String)
```

Busca de Dados

```
Future<List> getAll() async {  
    Database? dbLivro = await db;  
    if (dbLivro != null) {  
        List listMap = await dbLivro.query(LivroContract.LivroTable);  
        List<Livro> listLivros = [];  
  
        for (Map m in listMap) {  
            listLivros.add(Livro.fromMap(m));  
        }  
        return listLivros;  
    } else {  
        return [];  
    }  
}
```

Busca de Dados

```
Future<Livro?> getLivro(int id) async {  
    Database? dbLivro = await db;  
    if (dbLivro != null) {  
        List<Map> maps = await dbLivro.query(LivroContract.LivroTable,  
            columns: [  
                LivroContract.idColumn,  
                LivroContract.tituloColumn,  
                LivroContract.autorColumn,  
                LivroContract.anoPublicacaoColumn,  
                LivroContract.avalicaoColumn  
            ],  
            where: "${LivroContract.idColumn} = ?",  
            whereArgs: [id]);  
        if (maps.length > 0)  
            return Livro.fromMap(maps.first);  
        else  
            return null;  
    }  
    return null;  
}
```

Usando Banco de Dados na Widget

```
class _FormLivroBodyState extends State<FormLivroBody> {  
  
  final livroHelper = LivroHelper();  
  
  @override  
  Widget build(BuildContext context) {  
    return Form(  
      (...)  
      TextButton(  
        onPressed: () async {  
          if (_formKey.currentState!.validate()) {  
            Livro l = Livro(tituloController.text, autorController.text,  
              int.parse(anoController.text), rating);  
            livroHelper.saveLivro(l);  
            Navigator.pop(context);  
          }  
        },  
      ),  
    ),  
  }  
}
```

Bibliografia

- <https://docs.flutter.dev/cookbook/persistence/sqlite>
- <https://docs.flutter.dev/cookbook/forms/validation>